

# Relazione\_HW1

## Introduction

### Description

The purpose of this project was to decipher a ciphertext given by the professor using the frequency analysis technique. Given the fact that the text is in English we can try to count the frequency of every character in the ciphertext and match it with a character that has a similar frequency in normal English.

It seems a fairly easy challenge, but during the process, there are various things that might go wrong, starting from the fact that the average frequency of a character doesn't necessarily match the actual frequencies, so we can have different frequencies, and we might have to make some adjustments.

### Technologies

Trying to make my life easier I have chosen to use *Jupyter Notebook* for its ability to have different chunks of code that can be executed step-by-step. This property has been useful for having a log of what I have done. I also avoided executing the entire code every time I had to make a slight change.

### Sources

To find information about the frequencies of the letters I used [Wikipedia](#), it also gave me other information, such as the letter frequency as the first letter in the word.

## Actual Work

### Initialization

The first step was to copy the average frequency of each letter in a dictionary with the pair:

`<Letter>: <Frequency>`, with the same structure I copied the first letter frequency.

I copied the ciphertext into a string called `ciphertext`.

The first thing to do was to simply count the occurrence of each letter and normalize it over the total length of the ciphertext.

In this first stage I assumed that the space character was not encoded, clearly that was a wrong assumption and I knew it was, but I wanted to believe this was a simple task.

```
def count_letters(chipertext):
    frq = {}
    leng = len(chipertext)
    for a in chipertext:
        if a in frq:
            frq[a] += 1
        else:
            frq[a] = 1
    leng = leng - frq[' '] - frq['\n'] #not counting spaces
    del frq[' ']
    del frq['\n']
    for a in frq:
        frq[a] = float('%.3f'%(frq[a]/leng*100))
    return frq
```

## Map creation

The first step after calculating the frequency is to create a map, every character in the ciphertext corresponds to a character in the plaintext.

Again, I used the dictionary as data structure.

```
def make_mp(list1, list2):
    '''returns the mp from list1 to list2'''
    mp = {}
    cnt = 0
    for i in list1:
        mp[i] = list2[cnt]
        cnt+=1
    return mp
```

Given two arrays the function creates a map element to element following the order, so the arrays needs to be ordered.

The function that translate the ciphertext is simple as well.

```
def decipher(mp, ciphertext):
    '''given the mp it returns the dechiper text'''
    decipher=""
    for a in ciphertext:
        if a in mp.keys():
            decipher += mp[a]
        else:
            decipher += a
    return decipher
```

## First attempt

The previous steps inevitably led to a fail, the ciphertext didn't have the minimum meaning to me, there was not a single word that looked similar to English.

This brought me to try to look at the ciphertext with more attention and I noticed something really strange, every phrase ended with the char *TT*.

So I tried to make a simple map `'T': ' '`, I simply replaced *T* with spaces, this looked like something different and more similar, looking at the length of the words, to normal English. Clearly that map wasn't right because I lost the character that was mapped by the space in ciphertext, so I added the following map `' ': 'T'` to avoid losing some characters.

I noticed also that the ciphertext was composed by a total of 25 character excluding new line and spaces instead of 26, I decided to ignore the problem because I could manually fine tune later.

## Second attempt

Observing the new ciphertext, that I used as base from that point, I noticed that I could clearly see by eyes some words like: *OUE*, *IAB*, *OUIO*, I imagined that *OUE* could be *THE* and *OUIO* could be *THAT*.

I decided to do the same thing as the first attempt, I used `count_letters` to have the frequency of each letter of the new ciphertext and so I made a first map.

This time I obtained some results, the *T* and the *E* were mapped correctly, so i could understand some words, I manually inserted some other map entries and I could finally understand the first phrase: "*THE TRAVELER LEFT HIS VILLAGE AT DAWN WITH A SMALL BAG ON HIS SHOULDER*".

From that it was just a matter of understanding some other words.

I used some helper function I created:

```
def is_coherent(m):
    new_m = set()
    for k,v in m.items():
        if v in new_m:
            print(f"mpping not coherent {v} overlaps")
        else:
            new_m.add(v)
    print("mp coherent")
```

The purpose of this function is to check if there is a character that is mapped multiple times, this would break the ciphertext.

```
def ins_map(a,b,map):
    '''the letter a becomes the letter b and vice-versa in the map mantaining
    it coherent'''
```

```

ka = 0
kb = 0
for k, v in map.items():
    if v == a:
        ka = k
    if v == b:
        kb = k
    if kb != 0 and ka != 0:
        break
    if kb == ka and kb != 0:
        print("swapping same letter!!")
        return
mp[ka] = b
mp[kb] = a

```

This function swaps the map of two letters `a` and `b`.

## Plaintext

Putting everything together I obtained the following plaintext:

THE TRAVELER LEFT HIS VILLAGE AT DAWN WITH A SMALL BAG ON HIS SHOULDER  
AND HE FOLLOWED THE NARROW ROAD THAT LED ACROSS THE FIELDS WHERE THE  
GRASS  
WAS STILL WET WITH DEW AND THE AIR WAS COOL WHILE THE SOUND OF BIRDS  
ECHOED FROM THE TREES THAT STOOD AT THE EDGE OF THE FOREST AND THE SKY  
GREW BRIGHTER WITH EVERY STEP AS THE SUN ROSE BEHIND THE DISTANT HILLS  
AND THE SHADOWS SHORTENED WHILE THE COLORS OF THE EARTH CHANGED  
SLOWLY  
FROM GREY TO GOLD AS IF THE LAND ITSELF WERE AWAKENING TO GREET HIM ON  
HIS JOURNEY THAT SEEMED TO STRETCH ENDLESSLY BEFORE HIM LIKE A RIVER  
WHOSE COURSE WAS UNKNOWN YET PROMISED TO LEAD SOMEWHERE BEYOND THE  
HORIZON  
WHERE THE CITY HE DREAMED OF STOOD WITH WALLS OF STONE AND TOWERS THAT  
GLITTERED IN THE LIGHT OF THE MORNING SUN THAT SHONE WITH A BRILLIANCE  
THAT MADE HIM THINK OF STORIES HE HAD HEARD AS A CHILD WHEN HE SAT BY  
THE FIRE AND LISTENED TO HIS MOTHER SPEAK OF HEROES WHO TRAVELED FAR  
AND FOUND WONDERS THAT CHANGED THEIR LIVES FOREVER WHILE HE WONDERED IF  
HIS OWN STEPS WOULD ONE DAY BRING HIM TO SUCH PLACES OR IF THE ROAD  
WOULD SIMPLY CONTINUE WITHOUT END ACROSS VALLEYS AND OVER RIVERS AND  
THROUGH FORESTS THAT GREW DARKER AS THE LIGHT OF DAY FADED AND THE MOON  
APPEARED IN THE SKY TO WATCH OVER HIM WITH SILENT EYES THAT SEEMED TO  
GUIDE HIS FEET ALONG PATHS HE HAD NEVER KNOWN EXISTED UNTIL THE NIGHT

OPENED THEM BEFORE HIM AND THE STARS ABOVE MARKED HIS WAY WITH THEIR DISTANT LIGHT THAT GLITTERED LIKE SILVER DUST SCATTERED ACROSS A BLACK CLOTH THAT COVERED THE HEAVENS AND MADE HIM FEEL BOTH SMALL AND YET PART OF SOMETHING GREATER THAN HIMSELF AS IF THE UNIVERSE HAD BEEN WAITING FOR HIM TO WALK THIS VERY ROAD AND TO LEARN THAT THE JOURNEY WAS NOT ONLY ABOUT REACHING THE CITY HE SOUGHT BUT ABOUT DISCOVERING THE STRENGTH THAT LAY WITHIN HIM AND THE COURAGE THAT GREW WITH EACH STEP THAT HE TOOK INTO THE UNKNOWN WHERE THE PROMISE OF TOMORROW WAITED LIKE A LIGHT THAT COULD NEVER BE EXTINGUISHED NO MATTER HOW LONG THE NIGHT OR HOW FAR THE ROAD THAT LED HIM EVER ONWARD INTO THE DISTANCE

## Source code

The Source code is available on [https://github.com/tizianocoluzzi/homework\\_cybersec.git](https://github.com/tizianocoluzzi/homework_cybersec.git), it's chaotic because is full of things I didn't use, e.g. the most common bigrams and trigrams in the text, I tried to take everything I could and use it after.